Science of Computer Programming 248 (2026) 103387

Contents lists available at ScienceDirect

Science of Computer Programming

journal homepage: www.elsevier.com/locate/scico

Enhancing automated loop invariant generation for complex programs with large language models

Ruibang Liu, Minyu Chen, Ling-I Wu, Jingyu Ke, Guoqiang Li *

Shanghai Jiao Tong University, 200240, Shanghai, China

ARTICLE INFO

Keywords:
Loop invariant generation
Static analysis
Large language models

ABSTRACT

Automated program verification has always been an important component of building trustworthy software. While the analysis of loops remains a theoretical challenge, the automation of loop invariant analysis has effectively resolved the problem. However, existing invariant generation tools are predominantly effective for programs with purely numerical or purely pointer-based structures. Real-world programs often mix complex data structures and control flows. These structures can include arrays, pointers, and recursive definitions, while control flows may involve multiple nested or concurrent loops. Traditional methods generally only generate invariants for simple numerical programs or specific segments, lacking broad applicability. In order to automatically generate loop invariants for real-world programs, we proposed *ACInv*, an Automated Complex program loop Invariant generation tool, which combines static analysis with prompting with Large Language Models (LLM) to generate the proper loop invariants. We employ static analysis to systematically decompose the program's data structures and loops. This involves layer-by-layer transmission of structural information about variables, numerical data, and the complete loop structure to the LLM, enabling the generation of corresponding invariants. In comparison to prior work on AutoSpec, we delve deeper into the variable information within each loop. We conducted experiments on ACInv, which showed that ACInv outperformed previous tools on data sets with data structures and maintained similar performance to the state-of-the-art tool AutoSpec on numerical programs without data structures. For the total data set, ACInv can solve 21% more examples than AutoSpec, and can generate reference data structure templates.

1. Introduction

Program verification has progressively emerged as a fundamental component in the development of reliable software within the industry, thereby necessitating advanced tools for automated program verification. Nonetheless, the verification of program properties in the presence of loops is inherently undecidable. Within the framework of Hoare logic [1], *loop invariants*, which are usually assertions that are always true during the entry and execution of the loop, serve as critical abstractions of loop properties. These invariants are indispensable in enabling verification tools to rigorously establish the correctness of programs. However, in the practical implementation of program verification tools such as CPAchecker [2], CBMC [3], SMACK [4] and Frama-C [5], the derivation of loop invariants often necessitates manual intervention by domain experts, which poses a substantial impediment to the full automation of program verification.

* Corresponding author.
E-mail address: li.g@sjtu.edu.cn (G. Li).

https://doi.org/10.1016/j.scico.2025.103387
Received 17 February 2025; Received in revised form 24 August 2025; Accepted 27 August 2025

Available online 29 August 2025
0167-6423/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Loop invariants are usually defined inductively, which means that they must meet the following two conditions: (1) **Initialization**: It is true before the first iteration of the loop; (2) **Maintenance**: If the invariant holds true at the beginning of an iteration, it must also hold true at the end of that iteration. From a theoretical perspective, for any real-world program containing loops, irrespective of the programming language or the complexity of the program's code structure or data structure, we aim to automatically generate correct loop invariants. Moreover, these loop invariants should be strong enough to support the verification of relevant properties under various conditions.

Although there is a substantial body of prior work on the automatic generation of loop invariants, these methods often face challenges when applied to the verification of real-world programs, primarily due to their generality and associated performance overhead. Abstract interpretation based methods [6–8] was initially adopted by researchers. These approaches typically employ static analysis techniques, such as value analysis and interval analysis, to trace the values manipulated by the program and subsequently synthesize these values to construct loop invariants. However, these methods are limited to analyzing programs with simple numerical types and are less effective in handling complex loops, advanced data types and structures, and fail to adequately address intricate relationships between variables. Template-based methods [9–13] typically involve predefining invariants using SMT formulas, proposing parameterized invariant templates via uninterpreted functions or inequalities, and employing heuristic strategies to determine the parameters. While this approach can reveal more complex relationships between variables, it remains largely confined to linear numerical inequalities and imposes significant computational costs due to the introduction of SMT constraint solving. The widespread adoption of machine learning has led many researchers to propose learning-based methods [14–17], which infer suitable invariant templates through specific deductions and learn the corresponding parameters using various learning techniques to generate invariants. However, their effectiveness remains limited by the predefined templates and the quality of the training data.

In recent years, *Large Language Models* (LLMs), such as ChatGPT, have emerged as powerful tools for automating various generative tasks. Several studies [18,19] have demonstrated their strong performance in program-related reasoning tasks. Notably, recent works [20–24] have highlighted the potential of LLMs in generating loop invariants. Trained on vast datasets encompassing both code and natural language, LLMs possess a sophisticated understanding of code patterns and semantic relationships. They excel at recognizing complex structures, such as loops, and are capable of automating invariant generation across extensive codebases, significantly reducing the need for manual effort.

Despite the potential of LLMs in invariant generation, several challenges remain. First, as highlighted by prior studies [25–28], the inferential capabilities of LLMs require further exploration, particularly through carefully crafted prompts. Second, LLMs may produce incorrect invariants, and these errors can compound when the models are repeatedly invoked. To mitigate this issue, researchers have integrated SMT solvers or verification tools into the workflow to filter out erroneous invariants. However, this solution introduces significant performance overhead, and its effectiveness hinges on the strength of the verification tool. Moreover, certain correct yet complex loop invariants may be rejected by the tool, necessitating additional manual intervention, which conflicts with the goal of an end-to-end approach. Lastly, even when error-free, the invariants generated by LLMs may lack sufficient strength to capture precise properties, thereby failing to assist in verifying the target property.

To address these challenges, we propose *ACInv*, an Automated Complex program loop Invariant generation tool leveraging Large Language Models (LLMs) in conjunction with static analysis. In order for ACInv to be able to abstract data structures and reason about invariants, we need some necessary program information. The tool begins by extracting detailed information about the program's data structures and variable values within loops using static analysis techniques. ACInv then queries LLMs to generate appropriate predicate templates, which abstract potential data structure properties in the program. After this step, it inserts the predicates generated by LLMs as comments in the program, further analyzing variable information within loops and annotating those variables that undergo value changes in each iteration. For nested loops, the tool recursively queries LLMs, starting with the innermost loop.

In order to enable ACInv to determine the correctness of the invariants it generates and further enhance the invariants it generates, we check and enhance each invariant generated by ACInv. This step is completed by the evaluator and optimizer built by LLM. The evaluator first evaluates the correctness of the invariant and then passes the evaluation results and the corresponding invariant to the optimizer for processing. For valid invariants, the optimizer seeks to refine them, while incorrect invariants are either weakened to align with the correct program states or discarded altogether.

We contend that *ACInv* offers several distinct advantages over existing tools. First, it is capable of analyzing a wide range of complex, real-world code, with its sophisticated template processing for data structures and effective handling of nested loops, enabling it to tackle intricately structured programs. Second, *ACInv* delivers a fully automated, end-to-end solution for loop invariant generation, significantly reducing the need for manual intervention throughout the process. Lastly, while maintaining a competitive level of correctness, *ACInv* minimizes time overhead and optimizes the quality of the generated invariants.

To evaluate ACInv, we conducted a series of experiments. We conducted experiments on ACInv on datasets with and without data structures. The experiments showed that ACInv performed better than all current tools on datasets with data structures. On datasets without data structures, ACInv also performed close to the performance of the state-of-art tool AutoSpec [22]. Overall, ACInv can solve 21.08% more examples than AutoSpec.

In summary, our contribution is:

- We have built a better automatic invariant generation analysis method based on LLM that can handle relatively complex data structures while maintaining a relatively high correctness rate with low local time consumption, and built a tool ACInv based on it.
- We generated some predicate templates that may be useful for some commonly used data structures and other user-defined data structures on benchmarks such as [23,29] and SV-COMP [30,31].

```
1 int main() {
2     int i = 0;
3     /*@
4      loop invariants 0 <= i <= 30;
5     */
6     while(i < 30){
7         ++i;
8     }
9     return 0;
10 }
```

Fig. 1. A simple C code example with loop invariants.

2. Background

2.1. Loop invariant

To provide a formal characterization of the invariant, we first abstract the loop and its invariants using Hoare logic [1] as follows:

$$\frac{\{I \wedge B\}\ S\ \{I\}}{\{I\}\ \textbf{while}\ (B)\ \textbf{do}\ (S)\ \{I \wedge \neg B\}}$$

Where we abstract the general loop structure in the form of pseudo-code into the expression **while** (B) **do** (S), defining B as the guard condition, S as the loop body, and I as the sought loop invariant. Given a loop invariant I, if the body S preserves I under the condition that both I and B hold, i.e., $\{I \wedge B\}\ S\ \{I\}$ is valid, then initiating the loop in a state satisfying I ensures that upon termination, the state satisfies both the invariant I and the negation of the guard condition $\neg B$, formally, $\{I\}$ **while** (B) **do** (S) $\{I \wedge \neg B\}$. This rule underscores the critical role of identifying a suitable loop invariant I for verifying loop behavior.

An example of loop invariants in ACSL The vast majority of invariant research is based on the C language, so this paper will also use C language code as the research object. In the following paper, we will use ANSI/ISO C Specification Language (ACSL) to describe the loop invariants of the C program, which can be automatically checked by the verification tool Frama-C [5] and its plugin *WP*. In ACSL, all the information needed is implemented as the annotation which begins with `\\@` or is wrapped with `\*@...*\`, where loop invariants are annotated beginning with `loop invariant`.

Consider the example depicted in Fig. 1. The provided C code snippet above demonstrates a simple while loop. The program starts by declaring an integer variable `i` and initializing it to `0`. The `while` loop runs as long as the condition `i < 30` holds true. During each iteration, the value of `i` is incremented by `1` (`++i`). When `i` reaches `30`, the condition `i < 30` becomes false, and the loop terminates. The C program implements a bounded iteration construct with a formally verified loop invariant $0 \leq \mathtt{i} \leq 30$ specified in ACSL annotations, where the loop initializes `i` to 0 and increments it via pre-increment operation $++\mathtt{i}$ until the guard condition $\mathtt{i} < 30$ becomes false. Through Hoare logic verification, the invariant satisfies initialization at entry ($\mathtt{i} = 0 \Rightarrow 0 \leq 0 \leq 30$), preservation under execution ($0 \leq \mathtt{i} \leq 30 \wedge \mathtt{i} < 30 \vdash 0 \leq \mathtt{i} + 1 \leq 30$ since $\mathtt{i} \leq 29$), and conclusion upon termination ($0 \leq \mathtt{i} \leq 30 \wedge \neg(\mathtt{i} < 30) \equiv \mathtt{i} = 30$).

What we still need to add and explain is, the invariant and predicate referenced herein, along with those mentioned subsequently, are incorporated into the original code in the form of annotations. However, these annotations differ significantly from conventional comments. Traditional comments are authored by programmers to articulate their design intentions informally. In contrast, predicates and invariants are employed to specify certain program properties rigorously; they serve to assist in program verification and similar tasks, and are expressed in a strict formal language based on the ACSL (ANSI/ISO C Specification Language). Consequently, they serve distinct purposes and are conventionally demarcated by specialized syntactic markers in practice.

2.2. Large language models (LLMs)

In recent years, the field of Natural Language Processing (NLP) has achieved significant advances. State-of-the-art models, particularly those built on the transformer architecture [32], have achieved remarkable success in addressing a wide range of natural language tasks. Large Language Models (LLMs), sophisticated neural networks trained on massive text corpora, have emerged as powerful tools for understanding and generating human-like language. By utilizing deep learning frameworks such as transformers, LLMs can effectively capture and model intricate linguistic patterns and relationships. These models represent a major leap forward in NLP, driven by the convergence of novel architectures, vast datasets, and enhanced computational resources. The development of large-scale pre-trained models like BERT (Bidirectional Encoder Representations from Transformers) [33] and GPT (Generative Pre-trained Transformer) [34] has led to substantial improvements across a variety of NLP tasks.

Prompting is a technique where specific input phrases or instructions, known as “prompts”, are used to guide the LLM’s responses. Prompting leverages the model’s pre-existing knowledge, allowing for immediate, on-the-fly task execution without the need for additional training. This method is highly flexible and efficient, especially for exploratory or dynamic tasks, as it does not require any modifications to the model’s internal parameters. Previous work has demonstrated that techniques such as Chain-of-Thought (CoT) prompting [26] and few-shot learning [34] are highly effective in enhancing the performance of prompts. CoT prompting encourages

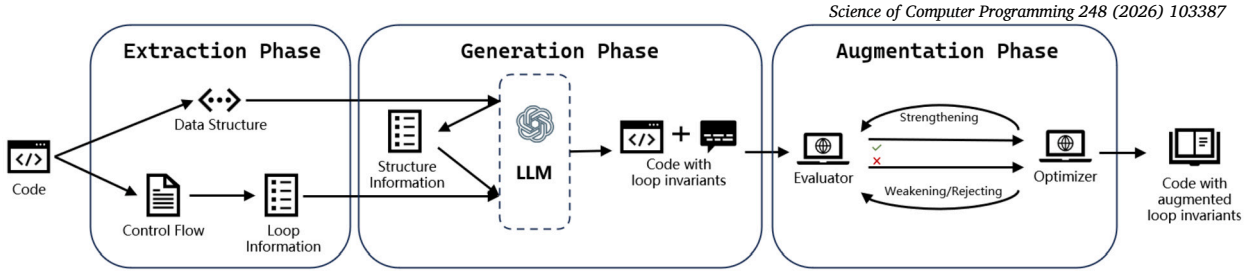


Fig. 2. The Workflow of ACInv.

the model to break down complex problems into intermediate reasoning steps, enabling more structured and logical responses. Few-shot learning involves providing the model with a small number of task-relevant examples within the prompt, which helps the model understand the specific format, context, or structure required for the task, even when it has not been explicitly fine-tuned for it. Drawing from various prompting techniques [25–28], we designed a structured prompt framework to help us enhance computational efficiency in deploying LLMs for inference.

3. Methodology

3.1. Overview

In Fig. 2, we present an overview diagram illustrating our proposed approach. To address the aforementioned challenge, we introduce ACInv, an LLM-based tool for automatic loop invariant generation. ACInv's process consists of three main phases: (1) **Extraction Phase**: ACInv initially extracts data structure and loop information from the target program. This involves performing basic control flow and value analyses, classifying loops, and gathering relevant variable information for each loop. (2) **Generation Phase**: During this phase, ACInv embeds the extracted data structure information into data structure prompts, querying LLMs to derive predicates that abstract key properties of the data structure. These predicates, along with loop-specific information, are then embedded into invariant prompts to generate loop invariants. If the current loop is not the first in the program, information from preceding loops, along with their generated invariants, will also be incorporated into the prompts. (3) **Augmentation Phase**: This section mainly consists of two components: the evaluator, and the optimizer, which are all composed of LLMs. The evaluator assesses the correctness of each generated invariant. For correct invariants, the optimizer attempts to strengthen them. For incorrect invariants, the optimizer weakens them to align with the correct program state. If an invariant cannot be refined to the correct level after k iterations, it is discarded. In the remainder of this section, we will provide a detailed explanation of the entire methodology and its related intricacies.

3.2. Problem description

To provide a more formal description of this problem, we formulate the invariants generation task as follows: let $\mathbb{M}$ denote the LLMs, and $\mathbb{P}$ represent the program with a maximum of n lines. Each line location in the code is denoted by $l_0, l_1, \cdots, l_n$, which may correspond to a line in a loop or data structure.

Let $d_{num} \in \mathcal{D}$ denote the indexed data structure instance retrieved by ordinal num, while $loop_{num} \in \mathcal{L}$ corresponds to an independent loop structure identified with the same indexing scheme, where $\mathcal{D}$ and $\mathcal{L}$ is the set of corresponding information. For each loop $loop_{num}$, assuming its specific position in the program $\mathbb{P}$ starts at line l_i, where $i \in \{1, n\}$, we define:

- $\mathbf{P}$ as the prompts, which mainly include four prompts, predicate prompt as $\mathbf{P}_{ds}$, invariant prompt as $\mathbf{P}_{inv}$, evaluator prompt as $\mathbf{P}_{ev}$, and optimizer prompt as $\mathbf{P}_{op}$,
- $\mathbb{M}(\mathbf{P}(loop_{num}, l_i, \mathbb{P}, \mathcal{D}))$ as the query result specific to $(loop_{num}, l_i)$, and
- $\{I_k^{num}\}_{k=0}^{m_{num}}$ as the set of loop invariants at l_i.

Therefore, the current problem becomes: Given a program $\mathbb{P}$ with a maximum number of lines n, for each loop $loop_{num} \in \mathcal{L}$ and its corresponding line l_i where $i \in \{1, n\}$, generate loop invariants $\{I_k^{num}\}_{k=0}^{m_{num}} \in \mathcal{I}$.

To formalize our requirements for the generated invariants' properties, we employ Fig. 3 to illustrate key characteristics. In this representation, distinct variables correspond to different dimensions. The value ranges of these variables and their interrelationships collectively delineate all reachable program states, denoted as PS in the figure. For simplicity, only two variables x and y are shown; practical scenarios may involve higher-dimensional relationships.

The invariant I we aim to generate typically constitutes an *over-approximation* of PS. Consequently, our primary requirement is *correctness*: the invariant must subsume all reachable program states, i.e., $PS \subseteq I$. Under this correctness guarantee, a *stronger* invariant is preferred, characterized by a tighter approximation closer to the true program state. Formally, smaller variable ranges yield stronger invariants. As illustrated, I_1 is stronger than I_0, whereas I_2 is weaker.

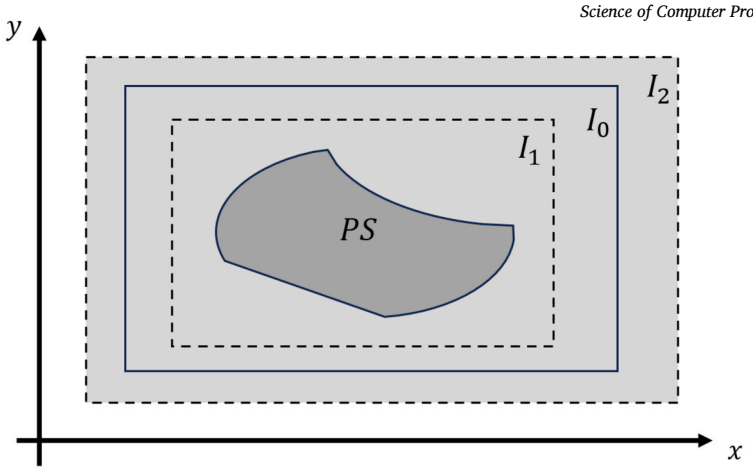


Fig. 3. Diagram of properties.

```
typedef struct VolDetectContext {
    uint64_t histogram[0x10001];
} VolDetectContext;

static int filter_frame(AVFilterLink *inlink, AVFrame *samples)
{
    AVFilterContext *ctx = inlink->dst;
    VolDetectContext *vd = ctx->priv;
    int nb_samples  = samples->nb_samples;
    int nb_channels = samples->ch_layout.nb_channels;
    int nb_planes   = nb_channels;
    int plane, i;
    int16_t *pcm;

    if (!av_sample_fmt_is_planar(samples->format)) {
        nb_samples *= nb_channels;
        nb_planes = 1;
    }

    for (plane = 0; plane < nb_planes; plane++) {
        pcm = (int16_t *)samples->extended_data[plane];
        for (i = 0; i < nb_samples; i++)
            vd->histogram[pcm[i] + 0x8000]++;
    }

    return ff_filter_frame(inlink->dst->outputs[0], samples);
}
```

Predicate:

```
/*@
  predicate valid_VolDetectContext(struct VolDetectContext* vd) =
    \valid(vd) && \valid(vd->histogram+(0 .. 0x10000));

  predicate non_negative_histogram(struct VolDetectContext* vd) =
    \forall integer i; 0 <= i <= 0x10000 ==> vd->histogram[i] >= 0;
*/
```

Loop No.1:

```
/*@ loop invariant 0 <= i <= nb_samples;
  @ loop invariant valid_VolDetectContext(vd);
  @ loop invariant non_negative_histogram(vd);
  @ loop invariant 0 <= pcm[i] + 0x8000 <= 0x10000;
*/
```

Loop No.2:

```
/*@ loop invariant 0 <= plane <= nb_planes;
  @ loop invariant valid_VolDetectContext(vd);
*/
```

Fig. 4. An illustrative example of C code from FFmpeg.

3.3. An illustrative example

Consider the example illustrated in Fig. 4. This program fragment is from the widely-used FFmpeg [35] library, an open-source multimedia framework written in C that facilitates the recording, conversion, and streaming of audio, video, and other multimedia files. For this program fragment, we will focus on its main components, omitting less relevant details.

The `VolDetectContext` structure is employed to store audio sample values for plotting frequency distributions. It contains a single member, `histogram[]`, which is an array of unsigned 64-bit integers used to represent the distribution of audio sample frequencies. The `filter_frame` function processes audio filters within FFmpeg, specifically handling audio frames and updating frequency distributions. It iterates through each audio plane of the input audio frame (`AVFrame`) and adjusts the corresponding count in `vd->histogram`, thus forming a nested loop structure. The expression `pcm[i] + 0x8000` shifts the sample values (typically ranging from -32768 to 32767) to a 0 to 0xFFFF range for indexing purposes. This illustrative case forms the basis for our theoretical extension in the following section, where we derive generalized insights through rigorous analytical decomposition.

3.4. Extraction phase

The primary objective of this phase is to extract structural and loop information from the target program $\mathbb{P}$. The input of this phase is a program or code segment $\mathbb{P}$, and output the data structure information $\mathcal{D} = \{d_{num}\}$ and loop information $\mathcal{L} = \{loop_{num}\}$. To achieve this, we implement a specialized parsing module that systematically traverses the entire program. The module first extracts data structure declarations and constructs a comprehensive data structure information list, denoted as $\mathcal{D} = \{d_{num}\}$. For each identified data structure, the module retrieves all its members and determines whether the data structure is recursive. A data structure is considered recursive if it contains a member of its own type. The recursiveness of a data structure significantly influences subsequent processing stages. At this phase, we merely flag the recursive nature of data structures, with detailed processing strategies to be elaborated in later stages.

More specifically, this stage actually achieves the following operations, such as Algorithm 1.

Algorithm 1: The algorithm for extraction phase.

Input : A C program $\mathbb{P}$
Output: Set $\mathcal{D}$ of data structures, Set $\mathcal{L}$ of loop information
$\mathcal{D} \leftarrow \emptyset$; // Initialize data structure set
$\mathcal{L} \leftarrow \emptyset$; // Initialize loop information set
$loopStack \leftarrow$ empty stack ; // Track nested loop positions
foreach *node n in the Abstract Syntax Tree (AST) of P* **do**
  if *n is a struct declaration* **then**
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(datatype, dataname))\}$; // Record struct
  else if *n is a loop (for/while/do-while)* **then**
    push n onto $loopStack$; // Update loop position
    $modifiedVars \leftarrow \emptyset$;
    foreach *variable v modified in n's body* **do**
      $modifiedVars \leftarrow modifiedVars \cup \{v\}$
    $loopDepth \leftarrow |loopStack|$ $\mathcal{L} \leftarrow \mathcal{L} \cup \{(modifiedVars, loopDepth)\}$; // Record loop
    pop n from $loopStack$
return $(\mathcal{D}, \mathcal{L})$;

This algorithm performs static analysis of C programs through abstract syntax tree (AST) traversal, extracting structural and behavioral metadata. The procedure takes a C program $\mathbb{P}$ as input and outputs two sets: $\mathcal{D}$ for data structure declarations and $\mathcal{L}$ for loop information. Initialization establishes empty sets $\mathcal{D}$ and $\mathcal{L}$, along with an empty loop stack to track nesting depth. During AST traversal, when encountering a struct declaration, the algorithm extracts its composite data type definition τ and adds τ to $\mathcal{D}$. For loop constructs (for/while/do-while), it pushes the loop node onto the stack, identifies all modified variables V_{mod} within the loop body, computes the current nesting depth as the stack size, records the tuple $(V_{\text{mod}}, \text{depth})$ to $\mathcal{L}$, and finally pops the loop node from the stack. The algorithm terminates by returning the tuple $(\mathcal{D}, \mathcal{L})$, providing comprehensive structural definitions and loop mutation patterns essential for downstream program analysis tasks such as parallelization or dataflow verification, with $\mathcal{O}(n)$ complexity proportional to AST node count.

Subsequently, ACInv systematically traverses the control flow graph of the program, with particular emphasis on loop structures. To facilitate structural analysis and provide clear decomposition pathways for the LLM, a hierarchical decomposition strategy is employed to break down complex loop constructs. During this process, ACInv identifies all loop instances within the program and assigns unique sequence numbers organized in a loop stack method. The numbering scheme follows these principles: for nested loops, the innermost loop receives the smallest number, while for parallel loops, priority is given to the loop appearing earlier in the program text. As illustrated in Fig. 4, the innermost loop is labeled as 1, the enclosing loop as 2, and if there is a subsequent loop following this nested structure, it would be assigned number 3. This sequential numbering scheme establishes an analytical framework that guides the program analysis in subsequent phases, ensuring systematic exploration of the program's control flow.

Variable information can help LLM focus its attention on key parts, providing assistance in generating invariants. For each loop, ACInv also captures key variable information, which is specifically, all assigned variables and parameters passed as pointers in function calls. The parser will provide all variables that have undergone value changes and determine their variable types, including whether they belong to the abstract data types defined by the previously extracted data structure. All the information, along with the loop sequence number, forms the loop structure information $loop_{num}$ and is then maintained in the list $\mathcal{L} = \{loop_{num}\}$.

For the example in Fig. 4, ACInv will record the declaration of the structure `VolDetectContext`, provide a judgment that it is not recursive, and then traverse the program body. After completing the traversal of the loop body, it assigns sequence number 1 to the inner loop with line numbers 23-24 and extracts key variables: `{i, vd}`; then assigns sequence number 2 to the outer loop with line numbers 20-25 and extracts key variables: `{plane, pcm[]}`, as well as `{i, vd}` inherited from the inner loop.

3.5. Generation phase

In this phase, ACInv will leverage the information extracted in the previous phase to generate loop invariants by querying LLMs. The generation module of ACInv accepts the input $(\mathbb{P}, \mathcal{D}, \mathcal{L})$, which is the target program and the information of data structure and loop parsed in the extraction module, and give the output $\{I'^{num}_k\}^m_{k=0}$ which is the preliminary invariant for each loop $loop_{num} \in \mathcal{L}$.

In real-world programs, a fundamental challenge in automated invariant generation stems from the prevalence of custom data structures that frequently appear in real-world programs. These structures often represent either specialized variations of conventional data structures or, more commonly, intricate compositions tailored to specific application requirements. To address this challenge, our approach emphasizes the systematic characterization and formal specification of the properties of these data structures as a prerequisite step prior to invariant generation. This preliminary analysis enables more accurate and meaningful invariant derivation that properly reflects the semantic complexity of real-world data organizations. First, ACInv will embed the data structure information $\mathcal{D} = \{d_{num}\}$ into the predicate prompts $\mathbf{P}_{ds}$, which is illustrated in Fig. 5 and query the LLM to obtain predicate templates that may be used for this data structure. In prompts, we emphasize that these predicates need to be able to abstract the data structure itself, and in addition, provide some other properties about this data structure.

Predicate prompts

```
Suppose you are an excellent C language
programmer and have some experience in formal
verification.

Given an C language program, you are supposed
to label some predicates in ACSL format for
the following data structure in C language
programs. You may need to generate the
following types of predicates:
1. The predicate that asserts some variable
represents a valid designative data
structure.
2. Some predicates that assert the properties
of this data structure.



To help you understand ACSL format, we
provide the following simple examples for
reference:

[[Examples]]



Here is our question, please judge whether it
is recursive and generate some predicates in
ACSL format for the following data structure:

[[Struct]]
```

Invariant prompts

```
Suppose you are an excellent C language
programmer and have some experience in formal
verification.

Given an C language program, you are supposed
to label the loop invariants at the position
specified by the placeholder "Loop" with the
ACSL format which only begins with "loop
invariant". Generally speaking, invariants
can be generated in the following way:
1. Analyze the range of values of key
variables;
2. Analyze the relationship between these
variables;
3. Analyze the relationship between these
variables and some involved constants;

To help you understand, we provide the
following examples:
please generate a loop invariant for the
following C code with a special attention to
variables listed in annotation:

[[Examples]]

Now please generate a loop invariant for the
following C code. Notice that we pay special
attention to variables listed in annotation:

[[Code]]
```

Evaluator prompts

```
Suppose you are an excellent C language
programmer and have some experience in formal
verification.

Given an C language program with some loop
invariants annotated, you are supposed to
judge whether the designative loop invariant
is correct, please think carefully, provide
reasons, and give me the final answer:

[[Examples]]
[[Code]]
```

Optimizer prompts

```
Suppose you are an excellent C language
programmer and have some experience in formal
verification.

Given an C language program with some loop
invariants annotated, we pointed out that an
invariant is correct/incorrect. If it is
correct, please strengthen it to try to
obtain a more refined invariant. If it is
incorrect, please weaken it to try to make
the invariant correct, and if it cannot be
weakened, please let me know to reject the
invariant:

[[Examples]]
[[Code]]
```

Fig. 5. The prompts used in ACInv.

```c
struct stack_int{
    size_t top;
    int data[MAX_SIZE];
}
```

(a) An non-recursive data structure example: stack.

```c
struct Node{
    int data;
    struct Node * next;
};
```

(b) A recursive data structure example: linked list.

Fig. 6. Two examples of data structure.

A critical observation reveals that the recursiveness of custom data structures constitutes a fundamental property that significantly influences both the formal specification of these structures in ACSL and the derivation of their associated properties. This characteristic plays a pivotal role in determining the appropriate representation and analysis of data structures throughout the verification process. In the syntax of ACSL, we can use `predicate` to define predicates related to non-recursive data structures, and `inductive` to define predicates related to recursive data structures. In order to help LLM understand predicate generation, we provided some examples to LLM, which are called shots in LLM prompting. The `[[Examples]]` section in the predicate prompts acts as a placeholder that will be dynamically filled with specific examples. Similarly, the `[[Code]]` placeholder will be replaced with the actual corresponding data structure. For non-recursively defined data structures, we provide a classic example: the stack, whose C language definition is usually as Fig. 6a.

In order to help LLM understand predicate generation, we provided some examples to LLM, which are called shots in LLM prompting. For non-recursively defined data structures, we provide a classic example: the stack, whose C language definition is usually as Fig. 6a.

We expect the generated predicate to at least express that this is an effective stack; In addition, we also hope that it can express some other properties about the stack, such as whether the stack is empty or full. Therefore, in this example, we provide the following predicate such as Fig. 7a. In ACSL grammar, `\valid(s)` means the pointer `s` is valid to read or write. The recursive data structure is usually related to pointers. Taking inspiration from separation logic [36], we provide such a recursive definition for the following single linked list, which considered both empty and non-empty linked lists, and expressed that the pointer `p1` to `p2` is a single linked list such as Fig. 7b.

Then, combined with the possible predicates, ACInv embeds the program $\mathbb{P}$ and the loop information $\mathcal{L}$ into the invariants prompt $\mathbf{P}_{inv}$ to query the LLM. Based on the order of the loops, ACInv will sequentially query the LLM for the loop with the smallest ordinal number, and require the LLM to generate invariants for that loop. Within each loop, we embed the processed program into prompts and require the LLM to analyze the following: (1) the range of values of the annotated variables; (2) Possible logical

```
/*@
  predicate valid_stack_int(struct stack_int* s) =
    \valid(s) && 0 <= s->top <= MAX_SIZE;

  predicate full_stack_int(struct stack_int* s) =
    \valid(s) && s->top == MAX_SIZE;

  predicate empty_stack_int(struct stack_int* s) =
    \valid(s) && s->top == 0;
*/
```

(a) The predicates of stack

```
/*@
 inductive is_linked_list{L}(struct Node *p1, struct Node *p2) {
      case empty{L}:
            \forall struct Node *p1, *p2;
              (p1 == p2) ==> is_linked_list(p1, p2);

      case non_empty{L}:
            \forall struct Node *p1, *p2;
              (\valid(p1) && is_linked_list(p1->next, p2))
                    ==> is_linked_list(p1, p2);
     }
*/
```

(b) The predicates of linked list

Fig. 7. Two examples of data structure.

relationships between these variables and constants; (3) Does the data structure variable satisfies the predicate. The `[[Example]]` section provides multiple program instances accompanied by their corresponding loop invariants, with particular emphasis on detailed explanations of nested loop structures and the associated data structures utilized within these examples. This section serves to illustrate both the general patterns and specific considerations for invariant generation in complex control flow scenarios and custom data structure implementations. From the responses generated by the LLM, we systematically extract the invariants corresponding to their designated positional indices. Specifically, for a given loop identified by its numerical label *num*, we obtain its corresponding invariant $\{I'^{num}_k\}^{m}_{k=0}$. When the traversal of all loops is completed, we obtain a comprehensive set of invariants, denoted as $\mathcal{I}' = \{\{I'^{1}_k\}^{m_1}_{k=0}, \{I'^{2}_k\}^{m_2}_{k=0}, \cdots, \{I'^{num}_k\}^{m_{num}}_{k=0}, \cdots\}$.

Fig. 4 illustrates the majority of the process for this step. Initially, two predicates are generated based on the data structure, which represents whether it is a valid `VolDetectContext` and the non-negativity of the array `histogram[]`. Using these predicates, ACInv first derives an invariant for Loop 1, which is an inner loop. Once this invariant is obtained, all invariants within this loop are forwarded to the next stage for enhancement. Following this, the enhanced invariant is used as a reference for generating the invariant for Loop 2 by querying the LLM.

3.6. Augmentation phase

prevalent challenge is that the invariants generated by LLMs often tend to be weak—for instance, trivial solutions such as $\mathcal{I} = TRUE$ are frequently produced. To address this issue while simultaneously filtering out incorrect candidates, we introduce an augmentation phase. In this phase, ACInv focuses on refining the generated invariants by either strengthening or weakening them. Building upon the previous phase where we derived the initial set of invariants $\mathcal{I}'$ from program $\mathbb{P}$, we now aim to perform a deeper analysis to refine and enhance the invariant collection. We seek to obtain an optimized set of invariants $\mathcal{I}$ that more accurately captures the program's behavioral properties and provides stronger verification guarantees. Unlike the previous phase, this stage is loop level, which means that for each individual loop, the generated invariant will first be sent to this stage to try to be enhanced, and then the enhanced invariant group will be sent back to the generation phase as a reference for generating the invariant in subsequent loops. For each loop $loop_{num}$, the i-th generated invariant I'^{num}_i will be enhanced independently.

It begins by utilizing an evaluator to assess the correctness of the invariants and then calls upon an optimizer to take appropriate actions based on the evaluation. Both the evaluator and optimizer are LLM-based. As illustrated in Fig. 5, we use elaborately designed dedicated prompts and examples to implement the LLM-based evaluator and optimizer. It is important to note that these components utilize LLMs independently, both with each other and with the LLM employed in the aforementioned generation phase. As mentioned earlier, we hope to use evaluators to ensure the correctness and optimizers to guarantee a stronger invariant. However, a very normal concern is that the LLM may make mistakes. However, relevant studies [20] have shown that the existing LLMs already have a strong enough reasoning ability and can give results with relatively reliable correctness rates to a certain extent. Through multiple independent information processing operations, we can further reduce its error probability. We expect to complete an end-to-end invariant generation work based on LLM, and thus it can be applied to scenarios that tolerate certain errors.

If an invariant is deemed correct, ACInv will attempt to "strengthen" it. In the context of invariants, this typically involves narrowing the range of program variable states represented, resulting in a more precise invariant. Conversely, if the invariant is deemed to be incorrect, ACInv seeks to "weaken" it. This process involves expanding the range of program variable states to encompass the correct set of states within the revised invariant. By broadening the scope of the incorrect invariant, ACInv aims to adjust it to better reflect the valid program states. However, our weakening process incorporates a rejection mechanism, allowing the LLM to decline the weakening attempt if it determines that the invariant cannot be properly weakened while maintaining correctness. Following this step, the results are subsequently returned to the evaluator for further validation. To control the iterative refinement process, we introduce a parameter k that limits the maximum number of query cycles. When this threshold is reached, invariants that remain incorrect are permanently rejected, while those that have been successfully validated are retained in the final invariant set. At this stage, we obtain the final set of invariants, denoted as $\mathcal{I} = \{\{I^{1}_k\}^{m_1}_{k=0}, \{I^{2}_k\}^{m_2}_{k=0}, \cdots, \{I^{num}_k\}^{m_{num}}_{k=0}, \cdots\}$.

In practice, before entering the augmentation phase, the invariant generated by ACInv for Loop 2 in Fig. 4 is `0 <= plane < nb_planes`. However, during the execution of the loop, although the outer loop condition is `plane < nb_planes`, the final iteration may satisfy `plane == nb_planes`. Thus, this invariant does not cover all possible program states. The evaluator can identify this issue and the optimizer can adjust the invariant by expanding its range to `0 <= plane <= nb_planes`, ultimately providing the refined invariant.

4. Experiment

4.1. Research question

In this section, we will propose several intriguing research questions concerning ACInv and conduct experiments to address these questions:

- **RQ1**: Can ACInv generate correct predicate templates for complex programs with data structure and thus generate correct loop invariants?
- **RQ2**: Can ACInv generate correct loop invariants for complex programs without data structure?
- **RQ3**: How useful is the augmentation phase for ACInv?

4.2. Experimental setup

Benchmark Our dataset is mainly divided into two parts: those with data structures and those without data structures. The dataset of data structure consists of three parts: LIG-MM [23,29], SV-COMP [30,31], and FFmpeg [35]. We will elaborate on these three datasets in more detail later. The dataset without data structure mainly comes from the benchmark from AutoSpec [22], and we will also briefly describe this dataset.

First, we selected the benchmark from LIG-MM, which is a new benchmark specifically for programs with complex data structures and memory manipulations. LIG-MM is also divided into four parts: (1) the course programs which originate from course homework; (2) SV-COMP [37], the competition on software verification; (3) Benchmark from SLING [38], which is a separation logic loop invariant inferring tool using dynamic analysis; (4) Some real-world programs from well-known software and systems. LIG-MM consists of C code and focuses on some complex data structures, especially the single-linked list, double-linked list, tree, and hash table, which can be an appropriate reference dataset. Due to the limit on the number of tokens, we selected 227 examples with relatively few lines of code for testing.

We also picked C programs from SV-COMP 2024. LIG-MM and the benchmark of AutoSpec both select some programs from SV-COMP. But unlike them, we chose different parts. We selected twenty-six programs, all of which include at least one loop and one data structure. Among these data structures, we excluded all structures containing the single-linked list, double-linked list, tree, and hash table to avoid duplication with the selected parts in LIG-MM. Therefore, these programs also include many different types of structures.

To evaluate the effectiveness of ACInv on real-world programs, we selected several program segments from the widely used FFmpeg, the audio and video processing library. Given the complexity of function calls and structures within C programs, as well as the extensive codebase, we focused on specific portions of the program. We applied simple partitioning techniques, ensuring that ACInv could concentrate on the relevant functions for analysis, and we picked the programs with at least one loop and at least one data structure.

We also selected programs from AutoSpec, an automated tool for generating ACSL specifications for C programs. AutoSpec's benchmark assesses the tool's ability to synthesize specifications, including loop invariants, preconditions, and postconditions of functions. Notably, some C programs in this benchmark do not contain loops, so we specifically chose a subset that includes loops for our testing.

Benchmark from AutoSpec comprises four parts, which we introduce briefly: (1) Frama-C problems [39], a repository focused on program verification challenges for Frama-C. We selected a portion containing C programs with loops. (2) SyGuS [40], which represents the invariants track from the Syntax-Guided Synthesis competition, featuring only programs with linear loop structures. (3) OOPSLA-13 [41,42], and (4) SV-COMP [37], both of which include benchmarks that may have nested or multiple loops. All these benchmarks consist of C code, primarily focused on numerical programs without explicit structural declarations.

Baseline In this study, we compare the performance of several tools with ACInv on datasets both with and without data structures. For datasets with data structures, we evaluate SLING, LLM-SE [23], GPT-4o, and AutoSpec. SLING is a dynamic analysis tool for inferring loop invariants based on separation logic. LLM-SE leverages a fine-tuned LLM (which is CodeGen [43] in the specific implementation) to generate separation logic loop invariants, incorporating predicate decomposition into its training. AutoSpec employs hierarchical specification generation to produce ACSL-style annotations. Both LLM-SE and AutoSpec utilize verification checkers to ensure the correctness of the generated invariants. GPT-4o, in this context, is used by directly embedding the code into prompts, asking GPT-4o-2024-08-06 to generate loop invariants without employing specific prompting techniques or examples.

For datasets without data structures, we assess CODE2Inv [16,44], CLN2Inv [14], GPT-4o, and AutoSpec. CODE2Inv and CLN2Inv are state-of-the-art invariant generation tools for numerical programs, based on graph neural networks and continuous logic networks, respectively. AutoSpec was originally a tool for testing on this benchmark, and we naturally referred to its results.

Environment We primarily employ *GPT-4o-2024-08-06* as the core model for implementing ACInv, with *GPT-3.5-turbo* serving as a control in our experiments. The experiments are performed on a system equipped with an AMD Ryzen 9 5950X CPU, running Ubuntu 23.04, 128GB of RAM, and two NVIDIA RTX 4090 GPUs, each with 24GB of memory.

4.3. RQ1: performance on structure datasets

Table 1 shows the main results of our experiments, where completion means the rate of programs whose loop invariants can be inferred correctly to complete the verification of this program. In this experiment, considering the balance between time and

Table 1
Comparison of Performance on Multiple Datasets.

Tools	Data Structure			non-Data Structure				Overall		
	LIG-MM	SVCOMP24	FFmpeg	Frama-C	SyGuS	OOPSLA	SVCOMP22	Struc	non-Struc	All
non-LLM										
SLING	.171	.000	.000	-	-	-	-	.128	-	.055
CODE2Inv	-	-	-	.000	.549	.196	.000	-	.358	.204
CLN2Inv	-	-	-	.000	**.932**	.000	.000	-	.541	.309
LLM-based										
LLM-SE	**.419**	-	-	-	-	-	-	.314	-	.135
GPT-4o	.054	.192	.059	.172	.278	.261	.143	.076	.249	.175
AutoSpec	.016	.308	.059	.414	.857	**.826**	**.762**	.064	**.786**	.476
ACInv-3.5	.139	.307	.118	.345	.714	.457	.429	.163	.589	.407
ACInv-4o	.318	**.462**	**.353**	**.483**	.797	.761	**.762**	**.343**	.747	**.574**

```
1     old = new_linked_list();
2     newn = NULL;
3     curr = old;
4
5     while(curr != NULL){
6         old = old -> next;
7         curr -> next = newn;
8         newn = curr;
9         curr = old;
10    }
```

Fig. 8. In-place reversal of the singly linked list.

performance, we set the augmentation iteration parameter k to 5. Compared to the traditional tool SLING, LLM-based technologies, including ACInv, demonstrate strong applicability. While SLING can only reason about the linked list portion of LIG-MM, it struggles with other complex data structures. Experimental results from LLM-SE indicate good performance on singly linked lists (sll), doubly linked lists (dll), trees, and hash tables. However, LLM-SE did not provide training sets or fine-tuned models, limiting the ability to evaluate its performance on other data structures. Additionally, LLM-SE incorporates pre-defined predicates during fine-tuning, enabling abstraction during reasoning, though this approach may restrict its effectiveness on other data structures or non-pointer data structures.

In comparison to pure LLM querying and the latest technology, AutoSpec, ACInv performs well on datasets involving data structures. This performance is anticipated. In pure LLM querying, we observed that while the LLM can recognize data structures in the program and analyze the associated variables, it struggles to abstract them into suitable logical forms. This often results in accurate formulas being expressed in natural language rather than logical terms. Regarding AutoSpec, its examples primarily focus on numerical types, leading to suboptimal performance on benchmarks involving data structures.

The FFmpeg dataset presents significant challenges due to its many unpredictable user-defined data structures, making it difficult for large models to abstract their properties. None of the tools performed well in this scenario, though ACInv exhibited some advantages. Overall, the experiments suggest that ACInv possesses a notable degree of abstraction and reasoning capability across various datasets involving complex data structures.

4.4. RQ2: performance on non-structure datasets

In numerical programs, ACInv demonstrated performance comparable to AutoSpec. The SyGuS benchmarks, which consist of relatively simple programs with a single loop and focus on basic numerical transformations, saw CLN2Inv achieving the best results. However, both CODE2Inv and CLN2Inv performed poorly on other datasets, likely due to limitations in their training sets, highlighting the broader applicability of LLM-based technologies. The Frama-C problems benchmark, which involves nested loops and more complex variable interactions, posed greater challenges. ACInv, leveraging value analysis, performed notably better in these cases.

Overall, ACInv approached state-of-the-art performance in numerical programs without data structures and demonstrated clear advantages across diverse datasets.

4.5. Case study

Considering the example as Fig. 8. This program snippet performs an in-place iterative reversal of a singly linked list through pointer manipulation. Three pointers are initialized: `old` references the head of the original list, `newn` is set to `NULL`, representing the nascent reversed list, and `curr` is initialized to traverse the original list starting at `old`. During each iteration, `old` is advanced to

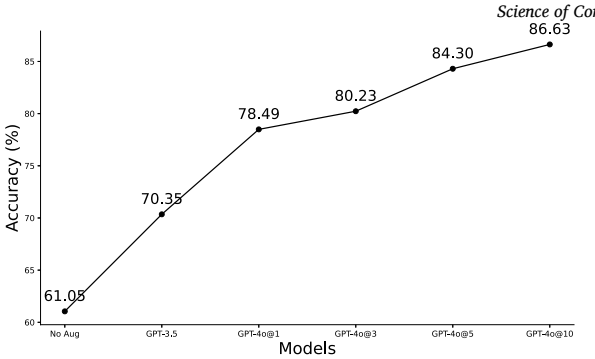


Fig. 9. Performance of Augmentation of ACInv.

its successor node to preserve traversal integrity, the `next` pointer of `curr` is redirected to `newn` to reverse linkage direction, `newn` is updated to reference the current node, thus extending the reversed sublist, and `curr` is reset to `old` for subsequent processing. The loop terminates when `curr` becomes `NULL`, at which point `newn` references the head of the fully reversed list. To give a specific example, if this linked list was originally `A->B->C->NULL`, it becomes `C->B->A->NULL` after the reversal.

Most of the other methods based on LLM can only obtain one invariant for this loop, namely `old == curr`. By observing the detailed output of the LLM, we can find that the LLM can actually capture some more specific properties. That is, this linked list is divided into two parts from the pointers old and curr. The part before curr is the flipped linked list, while the part after old is the linked list that has not been flipped yet. However, LLMs cannot describe this property completely formally. Instead, it describes it in a semi-formal and semi-natural language way, such as *`curr == suffix` of the original list starting at position `i`*. And our work can provide a more formal description of this nature, namely `is_linked_list(curr,NULL)`, `is_linked_list(old,NULL)`.

4.6. RQ3: testing for the augmentation

Fig. 9 shows the results of the experiment on the augmentation of ACInv. We tested the invariants generated by ACInv on a structured dataset, where Acc means the proportion of generated invariants that can pass the frama-c check, and model@k means k augmentation iterations were performed on the model. The experiment shows that when we use appropriate prompts and examples, the augmentation stage can judge the correctness of the invariants with a relatively high probability and filter out incorrect invariants.

4.7. Threats to validity

Data leakage. The extensive training sets used in LLMs can lead to potential data leakage, meaning that some of the code related to the questions we inquired about might have appeared in the training data of certain models. Although we have applied simple variable renaming and reordered the code, similar issues may still arise.

Correctness determination. Most related work, particularly during the final evaluation stage (and in some cases even in earlier stages), employs a deterministic checker to validate the generated invariants for filtering purposes. We did not adopt this approach here, partly due to performance overhead concerns and partly to avoid making the generated invariants dependent on a specific checker. In our experiments, we observed that some correct invariants could not be verified by the checker due to limitations in the checker's capability and available computational resources. In future work, we aim to follow up on this aspect, seeking to strike a more effective balance between correctness and scalability.

5. Related work

5.1. LLM for static analysis

Recent advances in LLMs have opened new avenues in static analysis. By integrating LLMs with traditional static analysis techniques, their scope can be broadened, leading to more effective and deeper program comprehension.

LLM4SA [45] leverages LLMs to analyze static warnings, reducing the need for manual effort. LLift [18] integrates static analysis with LLM-based prompting, successfully identifying Use Before Initialization (UBI) bugs in the Linux kernel. CFStra [19] employs LLMs to automatically select verification strategies based on code features and specifications. IRIS [46] combines LLMs with static analysis to perform whole-repository reasoning, detecting security vulnerabilities. NS-Slicer [47] uses a fine-tuned model to predict static program slices for both complete and partial code. Codeplan [48] synthesizes a multi-step chain of edits to assist with repository-level coding. ClarifyGPT [49] enhances code generation by leveraging LLMs, while AutoFL [50] utilizes LLMs to generate explanations of bugs for the purpose of Fault Localization (FL). Overall, incorporating LLMs has mitigated challenges related to applicability and performance overhead in static analysis and has enhanced the ability to analyze real-world code.

5.2. Invariants generation

Symbolic methods Traditional symbolic methods for invariant generation typically involve direct code analysis, mainly including abstraction interpretation and constraint solving. Abstract interpretation [51] approximates a program's behavior by mapping its concrete semantics to a simplified abstract domain, often using fixed-point calculations to abstract program states in the presence of loops. Enric et al. [6] developed a framework for generating conjunctions of loop invariants constrained to linear inequalities or polynomial equations, later extending the approach to polynomial invariants of bounded degree [7]. Steven et al. [8] applied a combination of Groebner bases, linear algebra, and abstract interpretation to generate polynomial invariants. Constraint-based methods, on the other hand, rely on pre-defined templates and employ SMT solvers to filter out appropriate invariant expressions. Farkas' lemma [9–13] is frequently used to extract linear relationships from loops, while other constraint templates, such as those based on Craig's interpolant [52,41,42], are also employed.

Separation Logic (SL) [36] is an extension of Hoare logic to reason about memory structures and the interactions of programs with dynamically allocated memory. It introduced the concept of "separating conjunction," which allows for concise reasoning about disjoint memory regions and parallel computation. Based on the extension of list manipulation [53], THOR [54–56] and MemCAD [57] are able to infer loop invariants on numeric and linked list programs. More recently, SLING [38] advances SL invariant generation by employing dynamic analysis to track memory and numerical changes in pointer-related structures.

Learning-based methods Learning-based methods often build upon template-based approaches by learning the parameters within these templates through various machine-learning techniques and ultimately invoking an SMT solver to filter suitable invariants. C2I [58] employs a randomized search to discover candidate invariants, followed by a checker to validate them. ICE [15] did learning for synthesizing invariants, leveraging examples, counterexamples, and implications. Then they combined them with decision trees [59]. CODE2INV [16,44] utilizes reinforcement learning and graph neural networks to learn program structures. LIPuS [17] also adopts reinforcement learning and uses pruning to reduce the search space. CLN2INV [14] introduces Continuous Logic Networks (CLN) to automatically learn loop invariants directly from program execution traces, while Gated-CLN (G-CLN) [60] extends this approach to allow the model to robustly learn general invariants across a large number of terms.

LLMs for loop invariants Just as we talked about above in the experiment section, LIG-SE [23] fine-tuned the large model and combined it with a checker, giving the LLM the ability to handle separation logic. It can also handle data structures and abstract predicate properties to a certain extent. Autospec [22] employs hierarchical specification generation to produce ACSL-style annotations, which include loop invariants and have achieved good performance in the processing of numerical programs. Both LLM-SE and AutoSpec call the checker at the end to ensure the correctness of the generated invariants. Several other works have explored the intersection of LLMs and loop invariant generation with promising outcomes. The iRank tool [20] introduces a re-ranking approach that evaluates loop invariants produced by LLMs. Unlike other methods, iRank does not focus on generating loop invariants but instead addresses the challenge of verifying the correctness of LLM-generated invariants. To achieve this, the authors employed two distinct LLM-based embedding models to learn the criteria for determining invariant correctness. Kexin et al. [21] aimed to fine-tune a pre-trained LLM for abstract reasoning over program executions, allowing for the generation of loop invariants. Their approach relies on a dynamic analyzer to trace program executions, which are then used to derive invariants. However, their evaluation is limited to a comparison against invariants generated by the dynamic analyzer, which poses inherent constraints. Loopy [24], on the other hand, continuously queries LLMs to generate invariants and employs a verification tool to ensure correctness. While they enhance the generated invariants by cyclically applying the Houdini algorithm and making repairs, the repeated invocation of the verifier introduces significant performance overhead.

6. Conclusion

In this paper, we proposed ACInv, an Automated Complex program loop Invariant generation tool leveraging Large Language Models (LLMs) in conjunction with static analysis. ACInv can abstract the data structure properties in complex programs and combine them to generate invariants. ACInv can also evaluate and enhance the generated invariants to a certain extent. We have conducted experiments on the effect of ACInv, and the experiments show that ACInv has achieved certain results on various data sets.

CRediT authorship contribution statement

Ruibang Liu: Writing – review & editing, Writing – original draft, Software, Formal analysis. **Minyu Chen:** Methodology, Conceptualization. **Ling-I Wu:** Methodology, Investigation. **Jingyu Ke:** Formal analysis, Data curation. **Guoqiang Li:** Writing – review & editing, Supervision, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Guoqiang Li reports was provided by Shanghai Jiao Tong University. Guoqiang Li reports a relationship with Shanghai Jiao Tong University that includes: employment. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] C.A.R. Hoare, An axiomatic basis for computer programming, Commun. ACM 12 (1969) 576–580, https://doi.org/10.1145/363235.363259.
[2] D. Beyer, M.E. Keremoglu, Cpachecker: a tool for configurable software verification, in: G. Gopalakrishnan, S. Qadeer (Eds.), Computer Aided Verification, Springer Berlin Heidelberg, Berlin, Heidelberg, 2011, pp. 184–190.
[3] D. Kroening, M. Tautschnig, Cbmc – c bounded model checker, in: E. Ábrahám, K. Havelund (Eds.), Tools and Algorithms for the Construction and Analysis of Systems, Springer Berlin Heidelberg, Berlin, Heidelberg, 2014, pp. 389–391.
[4] Z. Rakamarić, M. Emmi, Smack: decoupling source language details from verifier implementations, in: A. Biere, R. Bloem (Eds.), Computer Aided Verification, Springer International Publishing, Cham, 2014, pp. 106–113.
[5] F. Kirchner, N. Kosmatov, V. Prevosto, J. Signoles, B. Yakobowski, Frama-c: a software analysis perspective, Form. Asp. Comput. 27 (2015) 573–609, https://doi.org/10.1007/s00165-014-0326-7.
[6] E. Rodríguez-Carbonell, D. Kapur, An abstract interpretation approach for automatic generation of polynomial invariants, in: R. Giacobazzi (Ed.), Static Analysis, Springer Berlin Heidelberg, Berlin, Heidelberg, 2004, pp. 280–295.
[7] E. Rodríguez-Carbonell, D. Kapur, Automatic generation of polynomial invariants of bounded degree using abstract interpretation, in: Special Issue on the 11th Static Analysis Symposium - SAS 2004, Sci. Comput. Program. 64 (2007) 54–75, https://doi.org/10.1016/j.scico.2006.03.003, https://www.sciencedirect.com/science/article/pii/S0167642306001572.
[8] S. de Oliveira, S. Bensalem, V. Prevosto, Polynomial invariants by linear algebra, in: Automated Technology for Verification and Analysis, Springer International Publishing, Cham, 2016, pp. 479–494.
[9] S. Sankaranarayanan, H.B. Sipma, Z. Manna, Constraint-based linear-relations analysis, in: R. Giacobazzi (Ed.), Static Analysis, Springer Berlin Heidelberg, Berlin, Heidelberg, 2004, pp. 53–68.
[10] P. Yao, J. Ke, J. Sun, H. Fu, R. Wu, K. Ren, Demystifying template-based invariant generation for bit-vector programs, in: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, Luxembourg, Luxembourg, 2023, pp. 673–685.
[11] H. Liu, G. Li, Empirically scalable invariant generation leveraging divide-and-conquer with pruning, in: W.-N. Chin, Z. Xu (Eds.), Theoretical Aspects of Software Engineering, Springer Nature Switzerland, Cham, 2024, pp. 324–342.
[12] M.A. Colón, S. Sankaranarayanan, H.B. Sipma, Linear invariant generation using non-linear constraint solving, in: W.A. Hunt, F. Somenzi (Eds.), Computer Aided Verification, Springer Berlin Heidelberg, Berlin, Heidelberg, 2003, pp. 420–432.
[13] H. Liu, H. Fu, Z. Yu, J. Song, G. Li, Scalable linear invariant generation with Farkas' lemma, Proc. ACM Program. Lang. 6 (2022), https://doi.org/10.1145/3563295.
[14] G. Ryan, J. Wong, J. Yao, R. Gu, S. Jana, Cln2inv: learning loop invariants with continuous logic networks, in: International Conference on Learning Representations, OpenReview.net, Addis Ababa, Ethiopia, 2020, pp. 1–17, https://openreview.net/forum?id=HJlfuTEtvB.
[15] P. Garg, C. Löding, P. Madhusudan, D. Neider, Ice: a robust framework for learning invariants, in: A. Biere, R. Bloem (Eds.), Computer Aided Verification, Springer International Publishing, Cham, 2014, pp. 69–87.
[16] X. Si, H. Dai, M. Raghothaman, M. Naik, L. Song, Learning loop invariants for program verification, in: Advances in Neural Information Processing Systems, vol. 31, Curran Associates, Inc., Palais des Congrès de Montréal, Montréal CANADA, 2018, pp. 7762–7773.
[17] S. Yu, T. Wang, J. Wang, Loop invariant inference through smt solving enhanced reinforcement learning, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Association for Computing Machinery, New York, NY, USA, 2023, pp. 175–187.
[18] H. Li, Y. Hao, Y. Zhai, Z. Qian, Enhancing static analysis for practical bug detection: an llm-integrated approach, Proc. ACM Program. Lang. 8 (2024), https://doi.org/10.1145/3649828.
[19] J. Su, L. Deng, C. Wen, S. Qin, C. Tian, Cfstra: enhancing configurable program analysis through llm-driven strategy selection based on code features, in: Theoretical Aspects of Software Engineering, Springer Nature Switzerland, Cham, 2024, pp. 374–391.
[20] S. Chakraborty, S. Lahiri, S. Fakhoury, A. Lal, M. Musuvathi, A. Rastogi, A. Senthilnathan, R. Sharma, N. Swamy, Ranking LLM-generated loop invariants for program verification, in: Findings of the Association for Computational Linguistics: EMNLP 2023, Association for Computational Linguistics, Singapore, 2023, pp. 9164–9175, https://aclanthology.org/2023.findings-emnlp.614.
[21] K. Pei, D. Bieber, K. Shi, C. Sutton, P. Yin, Can large language models reason about program invariants?, in: Proceedings of the 40th International Conference on Machine Learning, in: Proceedings of Machine Learning Research, vol. 202, PMLR, Honolulu, Hawaii, USA, 2023, pp. 27496–27520, https://proceedings.mlr.press/v202/pei23a.html.
[22] C. Wen, J. Cao, J. Su, Z. Xu, S. Qin, M. He, H. Li, S.-C. Cheung, C. Tian, Enchanting program specification synthesis by large language models using static analysis and program verification, in: Computer Aided Verification, Springer Nature Switzerland, Cham, 2024, pp. 302–328.
[23] C. Liu, X. Wu, Y. Feng, Q. Cao, J. Yan, Towards general loop invariant generation: a benchmark of programs with memory manipulation, https://arxiv.org/abs/2311.10483, 2024, arXiv:2311.10483.
[24] A. Kamath, A. Senthilnathan, S. Chakraborty, P. Deligiannis, S.K. Lahiri, A. Lal, A. Rastogi, S. Roy, R. Sharma, Finding inductive loop invariants using large language models, https://arxiv.org/abs/2311.07948, 2023, arXiv:2311.07948.
[25] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, G. Neubig, Pre-train, prompt, and predict: a systematic survey of prompting methods in natural language processing, ACM Comput. Surv. 55 (2023), https://doi.org/10.1145/3560815.
[26] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q.V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), Advances in Neural Information Processing Systems, vol. 35, Curran Associates, Inc., Louisiana, United States, 2022, pp. 24824–24837, https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf.
[27] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, K. Narasimhan, Tree of thoughts: deliberate problem solving with large language models, in: A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, S. Levine (Eds.), Advances in Neural Information Processing Systems, vol. 36, Curran Associates, Inc., New Orleans, LA, USA, 2023, pp. 11809–11822, https://proceedings.neurips.cc/paper_files/paper/2023/file/271db9922b8d1f4dd7aaef84ed5ac703-Paper-Conference.pdf.
[28] C. Yang, X. Wang, Y. Lu, H. Liu, Q.V. Le, D. Zhou, X. Chen, Large language models as optimizers, in: The Twelfth International Conference on Learning Representations, OpenReview.net, Messe Wien Exhibition and Congress Center, Vienna Austria, 2024, pp. 1–41, https://openreview.net/forum?id=Bb4VGOWELI.
[29] Thinklab-SJTU, Lig-mm, https://github.com/Thinklab-SJTU/LIG-MM/tree/main, 2023. (Accessed 26 July 2024).
[30] D. Beyer, State of the art in software verification and witness validation: Sv-comp 2024, in: B. Finkbeiner, L. Kovács (Eds.), Tools and Algorithms for the Construction and Analysis of Systems, Springer Nature Switzerland, Cham, 2024, pp. 299–329.
[31] SoSy-Lab, Sv-comp 2024, https://gitlab.com/sosy-lab/benchmarking/sv-benchmarks/tree/svcomp24, 2024. (Accessed 6 July 2024).
[32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L.u. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems, vol. 30, Curran Associates, Inc., Long Beach, CA, USA, 2017, pp. 5998–6008, https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
[33] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), Association for Computational Linguistics, Minneapolis, Minnesota, 2019, pp. 4171–4186, https://aclanthology.org/N19-1423.

[34] T. Brown, B. Mann, N. Ryder, M. Subbiah, J.D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, in: Advances in Neural Information Processing Systems, vol. 33, Curran Associates, Inc., 2020, pp. 1877–1901, https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf, virtual.
[35] FFmpeg, Ffmpeg, https://github.com/FFmpeg/FFmpeg, 2023. (Accessed 26 July 2024).
[36] J. Reynolds, Separation logic: a logic for shared mutable data structures, in: Proceedings 17th Annual IEEE Symposium on Logic in Computer Science, IEEE, Copenhagen, Denmark, 2002, pp. 55–74.
[37] D. Beyer, Progress on software verification: Sv-comp 2022, in: Tools and Algorithms for the Construction and Analysis of Systems, Springer International Publishing, Cham, 2022, pp. 375–402.
[38] T.C. Le, G. Zheng, T. Nguyen, Sling: using dynamic analysis to infer program invariants in separation logic, in: Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019, Association for Computing Machinery, New York, NY, USA, 2019, pp. 788–801.
[39] M. Patnaik, Frama-c-problems, https://github.com/manavpatnaik/frama-c-problems, 2023. (Accessed 6 July 2024).
[40] R. Alur, D. Fisman, S. Padhi, R. Singh, A. Udupa, Sygus-comp 2018: results and analysis, https://arxiv.org/abs/1904.07146, 2019, arXiv:1904.07146.
[41] I. Dillig, T. Dillig, B. Li, K. McMillan, Inductive invariant generation via abductive inference, SIGPLAN Not. 48 (2013) 443–456, https://doi.org/10.1145/2544173.2509511.
[42] I. Dillig, T. Dillig, B. Li, K. McMillan, Inductive invariant generation via abductive inference, in: Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications, OOPSLA '13, Association for Computing Machinery, New York, NY, USA, 2013, pp. 443–456.
[43] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, Y. Zhou, Codegen2: lessons for training llms on programming and natural languages, https://arxiv.org/abs/2305.02309, 2023, arXiv:2305.02309.
[44] X. Si, A. Naik, H. Dai, M. Naik, L. Song, Code2inv: a deep learning framework for program verification, in: Computer Aided Verification, Springer International Publishing, Cham, 2020, pp. 151–164.
[45] C. Wen, Y. Cai, B. Zhang, J. Su, Z. Xu, D. Liu, S. Qin, Z. Ming, T. Cong, Automatically inspecting thousands of static bug warnings with large language model: how far are we?, ACM Trans. Knowl. Discov. Data 18 (2024), https://doi.org/10.1145/3653718.
[46] Z. Li, S. Dutta, M. Naik, Llm-assisted static analysis for detecting security vulnerabilities, https://arxiv.org/abs/2405.17238, 2024, arXiv:2405.17238.
[47] A. Yadavally, Y. Li, S. Wang, T.N. Nguyen, A learning-based approach to static program slicing, Proc. ACM Program. Lang. 8 (2024), https://doi.org/10.1145/3649814.
[48] R. Bairi, A. Sonwane, A. Kanade, V.D. C, A. Iyer, S. Parthasarathy, S. Rajamani, B. Ashok, S. Shet, Codeplan: repository-level coding using llms and planning, Proc. ACM Softw. Eng. 1 (2024), https://doi.org/10.1145/3643757.
[49] F. Mu, L. Shi, S. Wang, Z. Yu, B. Zhang, C. Wang, S. Liu, Q. Wang, Clarifygpt: a framework for enhancing llm-based code generation via requirements clarification, Proc. ACM Softw. Eng. 1 (2024), https://doi.org/10.1145/3660810.
[50] S. Kang, G. An, S. Yoo, A quantitative and qualitative evaluation of llm-based explainable fault localization, Proc. ACM Softw. Eng. 1 (2024), https://doi.org/10.1145/3660771.
[51] P. Cousot, R. Cousot, Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints, in: Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages, POPL '77, Association for Computing Machinery, New York, NY, USA, 1977, pp. 238–252.
[52] K.L. McMillan, Interpolation and sat-based model checking, in: Computer Aided Verification, Springer Berlin Heidelberg, Berlin, Heidelberg, 2003, pp. 1–13.
[53] S. Magill, A. Nanevski, E.M. Clarke, P. Lee, Inferring invariants in separation logic for imperative list-processing programs, in: Conference Record of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Association for Computing Machinery, New York, NY, USA, 2006, pp. 47–60, https://software.imdea.org/~aleks/papers/separation/separation.pdf.
[54] S. Magill, M.-H. Tsai, P. Lee, Y.-K. Tsay, Thor: a tool for reasoning about shape and arithmetic, in: Computer Aided Verification, Springer Berlin Heidelberg, Berlin, Heidelberg, 2008, pp. 428–432.
[55] S. Magill, M.-H. Tsai, P. Lee, Y.-K. Tsay, Automatic numeric abstractions for heap-manipulating programs, in: Proceedings of the 37th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '10, Association for Computing Machinery, New York, NY, USA, 2010, pp. 211–222.
[56] S. Magill, M.-H. Tsai, P. Lee, Y.-K. Tsay, Automatic numeric abstractions for heap-manipulating programs, SIGPLAN Not. 45 (2010) 211–222, https://doi.org/10.1145/1707801.1706326.
[57] H. Illous, M. Lemerre, X. Rival, A relational shape abstract domain, in: NASA Formal Methods, Springer International Publishing, Cham, 2017, pp. 212–229.
[58] R. Sharma, A. Aiken, From invariant checking to invariant inference using randomized search, in: Computer Aided Verification, Springer International Publishing, Cham, 2014, pp. 88–105.
[59] P. Garg, D. Neider, P. Madhusudan, D. Roth, Learning invariants using decision trees and implication counterexamples, SIGPLAN Not. 51 (2016) 499–512, https://doi.org/10.1145/2914770.2837664.
[60] J. Yao, G. Ryan, J. Wong, S. Jana, R. Gu, Learning nonlinear loop invariants with gated continuous logic networks, in: Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2020, Association for Computing Machinery, New York, NY, USA, 2020, pp. 106–120.